



You can ask the AI beast where it's vulnerable, and it will spit out a curated list of academic papers, industry guidelines, tools, and practical resources for evaluating and red teaming AI systems with a focus on jailbreaking and prompt injection. Look inside the large language model the AI is based on and you see nothing but tiny numbers, weights, and biases folded into tensuses useless to the naked eye. No scanner in the world can read a dump of those numbers and tell you whether a passport number or a confidential memo was in the training data. So, forget the fantasy of X-raying a model to find the secrets trapped inside. The only real method we have is simpler, dumber, and far more dangerous. Poke the model until it blurts something out. Which brings me to fine-tuning. Fine-tuning is the corporate equivalent of leaving sensitive documents lying around in a bar. You take your private training files, upload them to OpenAI or another provider, and ask them to train your personalized model. Now, your secrets exist in at least three copies locally, remotely, and in the mathematical skull of the model. And even if the model is trained to never reveal private data, that's just a loose behavioral hint, not a commandment. The model resists at first, like a dog trained not to steal from the counter. Just keep asking. Persistence over brilliance. And eventually the AI cracks. Passport numbers, phone numbers, biographical details. At first half right, then almost perfect. The randomness baked into every AI output means that if you roll the dice long enough, you hit THE JACKPOT. THAT'S NOT HACKING. That's waiting. And the same madness plays out in image generation. One moment the AI happily makes a meme for

you. The next it refuses and cites policy. You hit retry more times, than you'd like and never get an image again. But the question remains, why did it work once? That's the problem with AI as security infrastructure. 1% failure is 100% compromise. If a firewall let through 1% of packets, you drag the engineer outside and make them explain themselves. If fine-tuning is a data leak, then rag retrieval augmented generation is a completely busted pipe under your floorboards. Here's how rag works. You ask a question. In the background, the system quietly queries a database of your internal documents via embedding search. Everything relevant gets stuffed into the prompt. The model answers with your data. Sounds great. Also sounds like a subpoena waiting to happen because now sensitive documents, emails, financial files are silently copied into logs, prompts, caches, and vector stores everywhere the model needs them. Everywhere developers forget they put them, which makes pulling those secrets out trivial. Classic prompt. above attacks, translation attacks, markdown fencing, creative nonsense. The model blocks dozens of attempts. But the longer the conversation gets, the more context the system pours in, the weaker the guardrails become. Eventually, it cracks. It prints the system prompt, then the sensitive rag data, then the admin credentials, 100% hit on the synthetic database. We learned three things. Longer context, higher failure probability. System prompts protect nothing. Rag leaks like a sweating RV window in winter. Academic papers document the rag thief. 70% of a knowledge base automatically extracted using nothing but iterative prompting. And all of that was still just foreplay. The vector trap. Now we get to the real ghost in the machine.

Embeddings. When a document is fed into an embedding model, you don't get text back. You get a vector. Hundreds or thousands of tiny numbers representing the meaning of the passage. Developers treat these as harmless abstractions. One vector database CEO said, "Vectors are like hashes, safe even if stolen." Wrong. Laughably wrong. Because unlike a hash, embeddings can be inverted. You can take a vector, run it through an inversion model, then a correction loop, and reconstruct the original text with eerie accuracy. Private medical details resurrected from what most engineers think are meaningless decimals, names, diagnosis, amounts, dates. The inversion accuracy is close to 100%. So imagine your entire company file store, email system, HR database, all converted into embeddings for AI search. Now imagine those embeddings leaking. You don't have to imagine it. It's already happening. AI fishing, emails embedding hidden instructions into rag context, tricking the model into exfiltrating data harmlessly wrapped in markdown links. Modern AI systems multiply private data. They replicate it across logs, prompt histories, vector indexes, training files, caches, backups. If a normal system leaks like a faucet, AI systems leak like a fire hydrant hit by a truck. So, how do we defend ourselves? Three simple rules. Be suspicious of any AI feature that automatically slurps up your documents. The convenience tax is paid in exposure. Interrogate vendors like they owe you money. Ask how they handle training data, logs, embeddings, retention. Watch how fast they blink. encrypt to the application layer. Before data ever touches a database, vector store or model input field, the crypto landscape is mixed. Confidential compute enclaves, homomorphic

encryption, tokenization, distance preserving methods, all imperfect. All better than nothing. AI systems don't just use your private data. They multiply it, distribute it, and leave it lying around in places nobody watches. And the kicker, it's easy to exploit. Ridiculously easy. No Hollywood hackers required. Just simple prompts, open-source tools, and stubborn patience. The shadow data is real, the leaks are real, and the machine, our shiny industrial god, has no idea how much it remembers. So, there I am, wondering how many vectors of my own life were already drifting out there in the void.